our shielding · Your smart contracts, our shielding · Your smart c

shieldify

Blue Protocol
Depth

SECURITY REVIEW

Date: 13 April 2026

CONTENTS

1. **About Shieldify Security** **3**
2. **Disclaimer** **3**
3. **About Blue Protocol – Depth** **3**
4. **Risk classification** **3**
   - 4.1 Impact 3
   - 4.2 Likelihood 4
5. **Security Review Summary** **4**
   - 5.1 Protocol Summary 4
   - 5.2 Scope 4
6. **Findings Summary** **4**
7. **Findings** **5**

1. About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and have secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

2. Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3. About Blue Protocol - Depth

DEPTH is a soulbound NFT vault protocol on Abstract (zkSync L2). Every user receives a non-transferable NFT vault that continuously generates $DEPTH tokens — tradeable on Aborean DEX from day one.

The economic engine rewards reinvestment over selling. Users who go faster and deeper burn more tokens, generate more protocol revenue, and earn higher returns than users who simply dump. Speed is the revenue engine.

Part of the BLUE Protocol ecosystem. BLUE has a whale. Whales dive deep. "How deep do you go?"

Learn more about Depth's concept and the technicalities behind it: here

4. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4.1 Impact

- **High** - results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** - results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** - losses will be limited but bearable - and covers vectors similar to griefing attacks that can be easily repaired

4.2 Likelihood

- **High** – almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** – still relatively likely, although only conditionally possible
- **Low** – requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5. Security Review Summary

The security review lasted 5 days with a total of 80 hours dedicated to the audit by the Shieldify team.

Overall, the code is well-written. The audit report contributed by identifying two Medium and one Low severity issues. They're related to flawed state management and unsafe state transitions that can overwrite or invalidate user entitlements. During the audit, several others were identified, but they're not included in the report as the protocol acknowledges them as design choices.

The Blue team has done a great job with their test suite and provided support and responses to all of the questions that the Shieldify researchers had.

5.1 Protocol Summary

Project Name	Blue Protocol – Depth
Repository	depth-contracts
Type of Project	Vault, ERC-721, ERC-5192, ERC-20
Security Review Timeline	4 days
Review Commit Hash	f5b095bd211564157db09e90aaa22e4106481464
Fixes Review Commit Hash	4772dcf10ef26c9e351f7c87ae89f64df92436e8

5.2 Scope

The following smart contracts were in the scope of the security review:

File	nSLOC
EmissionController.sol	630
DepthToken.sol	225
DepthRaffle.sol	363
DepthSoul.sol	118
Total	**1336**

6. Findings Summary

The following number of issues have been identified, sorted by their severity:

- **Medium** issues: 2

- **Low** issues: `1`

ID	Title	Severity	Status
[M-01]	`selectWinner()` Can Overwrite a Live Higher-Tier Voucher With a Stale Lower-Tier Prize	Medium	Fixed
[M-02]	Owner-Controlled State Transitions Can Destroy Existing User Entitlements	Medium	Fixed
[L-01]	The `upgradeTier()` Uses the Vault's Frozen `totalCap` Instead of the Live Tier Config, so Lowering Any Tier Cap Permanently Bricks Upgrades for Existing Vaults	Low	Fixed

7. Findings

[M-01] `selectWinner()` Can Overwrite a Live Higher-Tier Voucher With a Stale Lower-Tier Prize

Severity

Medium Risk

Description

The `buyTickets()` validates raffle eligibility only at ticket-purchase time by mapping the buyer's then-current tier to `configIdByTier[currentTier]`. `selectWinner()` later trusts that historical `round.configId` and blindly writes `config.toTier` into the vault's upgrade-voucher slot without re-checking the winner's live tier.

That would already let the raffle mint a dead voucher when the selected winner has advanced since buying tickets. The more serious consequence is that `EmissionController` stores only a single `upgradeVouchers[tokenId]` value, and `setUpgradeVoucher()` overwrites it unconditionally. An older raffle round can therefore destroy a newer, still-redeemable voucher that the same vault earned after advancing tiers.

Example: a user buys Tier-0 -> Tier-1 raffle tickets, later upgrades to Tier 1 through `upgradeTier()`, and then obtains a valid Tier-1 -> Tier-2 voucher through `purchaseUpgrade()` or `claimGuaranteedUpgrade()`. If the old Tier-0 round is drawn afterwards and picks that user, `selectWinner()` overwrites the live Tier-2 voucher with `targetTier = 1`. The replacement voucher is immediately irredeemable because `redeemUpgradeVoucher()` requires `currentTier + 1 == targetTier`.

Location of Affected Code

`DepthRaffle.sol` - `buyTickets()` snapshots eligibility from the user's current tier:

File: DepthRaffle.sol#L184-L185

```
function buyTickets(uint256 count, uint256 tokenId) external
    whenNotPaused nonReentrant {
  // code
  uint8 currentTier = _getVaultTier(tokenId);
  uint256 configId = configIdByTier[currentTier];
  // code
}
```

`DepthRaffle.sol` – `selectWinner()` reuses the old config and finalizes the prize without re-checking the winner's live tier:

File: DepthRaffle.sol#L226-L246

```
function selectWinner(uint256 roundId, uint256 winnerIndex) external
    onlyOperator nonReentrant {
     RoundData storage round = rounds[roundId];
     if (round.configId == 0) revert InvalidRound(roundId);
     if (!round.filled) revert RoundNotFilled(roundId);
     if (round.drawn) revert RoundAlreadyDrawn(roundId);
     if (winnerIndex >= round.tickets.length) revert InvalidWinnerIndex(
        winnerIndex, round.tickets.length);

     RaffleConfig storage config = raffleConfigs[round.configId];
     address winner = round.tickets[winnerIndex];
     uint256 winnerTokenId = IDepthSoulRaffle(depthSoul).tokenIdOf(winner)
        ;
     if (winnerTokenId == 0) revert MissingSoul(winner);

     round.winner = winner;
     round.drawn = true;
     hasUpgraded[winner][round.configId] = true;
     IEmissionControllerRaffle(emissionController).setUpgradeVoucher(
        winnerTokenId, config.toTier);

     emit WinnerSelected(roundId, winner, winnerTokenId, winnerIndex);
     _openRound(round.configId, false);
}
```

`DepthRaffle.sol` – after the winner has already advanced, the same vault can legitimately acquire a newer voucher through another path:

File: DepthRaffle.sol#L263-L264

```
function claimGuaranteedUpgrade(uint256 tokenId) external whenNotPaused
    nonReentrant {
  // code
  hasUpgraded[msg.sender][configId] = true;
  IEmissionControllerRaffle(emissionController).setUpgradeVoucher(tokenId
      , config.toTier);

  emit GuaranteedUpgradeClaimed(msg.sender, tokenId, configId);
```

```
}
```

File: DepthRaffle.sol#L297-L300

```
function purchaseUpgrade(uint256 tokenId) external whenNotPaused
    nonReentrant {
     // code
     hasUpgraded[msg.sender][configId] = true;
     IEmissionControllerRaffle(emissionController).setUpgradeVoucher(
        tokenId, config.toTier);

     emit UpgradePurchased(msg.sender, tokenId, configId, cost);
}
```

`EmissionController.sol` — voucher storage is a single slot and writes are blind overwrites:

File: EmissionController.sol

```
mapping(uint256 => uint8) public upgradeVouchers;

function setUpgradeVoucher(uint256 tokenId, uint8 targetTier) external {
     require(msg.sender == raffleContract, "Not raffle contract");
     upgradeVouchers[tokenId] = targetTier;
     emit UpgradeVoucherSet(tokenId, targetTier);
}
```

`EmissionController.sol` — stale lower-tier vouchers cannot be redeemed:

File: EmissionController.sol#L473-L474

```
function redeemUpgradeVoucher(uint256 tokenId) external whenNotPaused {
  // code
  uint8 currentTier = vault.tier;
  require(currentTier + 1 == targetTier, "Voucher tier mismatch");
  // code
}
```

Impact

- A filled raffle round can burn its prize on a winner who is no longer eligible for that tier transition.
- More importantly, an old round can overwrite and destroy a newer, valid, higher-tier voucher already held by the same vault.
- The user can lose the full economic value of that later voucher path, whether it was obtained by direct upgrade purchase or by accumulating ticket credit for the higher tier.
- There is no on-chain merge, reroll, or refund path. Once the stale overwrite happens, the round is marked drawn and the previously valid voucher is gone.

Proof of Concept

1. Alice is at Tier 0 and buys tickets for the Tier-0 -> Tier-1 raffle. -> `buyTickets()` records her entries in the Tier-0 round.

2. Before that round is drawn, Alice calls `upgradeTier(tokenId)` -> Her vault advances to Tier 1.
3. While now at Tier 1, Alice acquires a valid Tier-1 -> Tier-2 voucher through `purchaseUpgrade(tokenId)` or `claimGuaranteedUpgrade(tokenId)` -> `upgradeVouchers[tokenId] = 2`
4. The old Tier-0 round later fills, and the operator selects Alice's ticket. -> `selectWinner()` uses the stale Tier-0 config -> `hasUpgraded[Alice][oldConfigId] = true` -> `setUpgradeVoucher(tokenId, 1)` -> `upgradeVouchers[tokenId]` is overwritten from 2 to 1
5. Alice calls `redeemUpgradeVoucher(tokenId)` -> `currentTier = 1, targetTier = 1` -> `require(currentTier + 1 == targetTier)` fails
6. The previously valid Tier-2 voucher has been destroyed, and the old raffle prize is useless as well.

Recommendation

The `selectWinner()` should validate the winner's live upgrade state before it consumes the round and writes any voucher. At minimum, the winner should still be at `config.fromTier`. If not, the function should not mark the round drawn for that ticket: it should either scan for the next eligible ticket or revert so the operator can provide a different winner index.

The contract should also prevent voucher downgrades. Because `upgradeVouchers[tokenId]` is a single slot, `selectWinner()` should read the current voucher first and refuse to overwrite an equal-or-better voucher that already exists. In practice, the round should only finalize when both of these conditions hold:

- `vault.tier == config.fromTier`
- `existingVoucher < config.toTier`

One concrete approach is:

```
interface IEmissionControllerRaffle {
    struct VaultData {
        uint8 tier;
        uint256 totalCap;
        uint256 totalClaimed;
        uint256 lastClaimTimestamp;
        uint256 unclaimedBuffer;
        uint256 cycleNumber;
        uint256 depthSacrificeCredit;
        uint256 gblueSacrificeCredit;
        uint256 rawGblueSacrificed;
        uint256 lastCheckIn;
        uint16 currentStreak;
        uint8 lockedGblueSacrificeLevel;
        bool resonanceActivated;
    }

    function vaults(uint256 tokenId) external view returns (VaultData
        memory);
```

```
    function upgradeVouchers(uint256 tokenId) external view returns (
        uint8);
    function setUpgradeVoucher(uint256 tokenId, uint8 targetTier)
        external;
}

function selectWinner(uint256 roundId, uint256 winnerIndex) external
    onlyOperator nonReentrant {
    RoundData storage round = rounds[roundId];
    RaffleConfig storage config = raffleConfigs[round.configId];

    address winner = round.tickets[winnerIndex];
    uint256 winnerTokenId = IDepthSoulRaffle(depthSoul).tokenIdOf(winner)
        ;
    if (winnerTokenId == 0) revert MissingSoul(winner);

    IEmissionControllerRaffle.VaultData memory vault =
        IEmissionControllerRaffle(emissionController).vaults(
            winnerTokenId);
    if (vault.tier != config.fromTier) revert WinnerNoLongerEligible(
        winner, vault.tier, config.fromTier);

    uint8 existingVoucher = IEmissionControllerRaffle(emissionController)
        .upgradeVouchers(winnerTokenId);
    if (existingVoucher >= config.toTier) {
        revert ExistingBetterOrEqualVoucher(winnerTokenId,
            existingVoucher, config.toTier);
    }

    round.winner = winner;
    round.drawn = true;
    hasUpgraded[winner][round.configId] = true;
    IEmissionControllerRaffle(emissionController).setUpgradeVoucher(
        winnerTokenId, config.toTier);

    emit WinnerSelected(roundId, winner, winnerTokenId, winnerIndex);
    _openRound(round.configId, false);
}
```

Team Response

Fixed.

[M-02] Owner-Controlled State Transitions Can Destroy Existing User Entitlements

Severity

Medium Risk

Description

This finding concerns two owner-controlled transition paths that share the same failure pattern: the protocol changes control-plane state without first preserving or settling the value users already earned under the previous state.

The first manifestation is in `resolveChallenge()`. A challenged tier-0 vault cannot claim rewards while `vaultStatus != 0`, but when the user later satisfies the ticket threshold and resolves the challenge, the function clears `vaultStatus` and resets reward accounting directly. Because it does not checkpoint pending rewards first, already-earned rewards can be erased at the moment the user regains access.

The second manifestation is in raffle-contract rebinding. `EmissionController` authorizes only one live `raffleContract` address for voucher issuance. When the owner updates that pointer, the previous raffle contract immediately loses authorization to call `setUpgradeVoucher()`. Any legacy raffle state that still depends on voucher issuance, including filled-but-undrawn rounds and old-contract voucher paths, becomes stranded even though users have already spent DEPTH or accumulated upgrade rights there.

Both paths require an owner-triggered transition, which keeps the severity in Low territory, but the resulting user-facing loss is still concrete once the transition occurs.

Location of Affected Code

Reward-wipe path:

File: EmissionController.sol

```
function claimRewards(uint256 tokenId) public whenNotPaused returns (
    uint256 claimedAmount) {
    VaultData storage vault = _requireInitializedVault(tokenId);
    address vaultOwner = _requireVaultOwner(tokenId);

    if (vault.tier == 0 && vaultStatus[tokenId] != 0) {
        return 0;
    }

    // code
}

function resolveChallenge(uint256 tokenId) external whenNotPaused {
    VaultData storage vault = _requireInitializedVault(tokenId);
    // code
    vaultStatus[tokenId] = 0;
    vault.lastClaimTimestamp = block.timestamp;
    vault.unclaimedBuffer = 0;

    emit ChallengeResolved(tokenId, vaultOwner);
}

function _checkpointPendingRewards(VaultData storage vault) internal {
    vault.unclaimedBuffer = _getPendingRewards(vault);
```

```
    vault.lastClaimTimestamp = block.timestamp;
}
```

Raffle-rebinding path:

File: EmissionController.sol

```
function setRaffleContract(address newRaffleContract) external onlyOwner
    {
    if (newRaffleContract == address(0)) revert InvalidAddress();
    raffleContract = newRaffleContract;
    emit RaffleContractSet(newRaffleContract);
}

function setUpgradeVoucher(uint256 tokenId, uint8 targetTier) external {
    require(msg.sender == raffleContract, "Not raffle contract");
    upgradeVouchers[tokenId] = targetTier;
    emit UpgradeVoucherSet(tokenId, targetTier);
}

function selectWinner(uint256 roundId, uint256 winnerIndex) external
    onlyOperator nonReentrant {
    // code
    IEmissionControllerRaffle(emissionController).setUpgradeVoucher(
        winnerTokenId, config.toTier);
    // code
}

function claimGuaranteedUpgrade(uint256 tokenId) external whenNotPaused
    nonReentrant {
    // code
    IEmissionControllerRaffle(emissionController).setUpgradeVoucher(
        tokenId, config.toTier);
}

function purchaseUpgrade(uint256 tokenId) external whenNotPaused
    nonReentrant {
    // code
    IEmissionControllerRaffle(emissionController).setUpgradeVoucher(
        tokenId, config.toTier);
}
```

Impact

This is not a permissionless exploit. Both branches depend on an owner action that changes the protocol state.

That owner constraint is why Low is the right severity. However, if the transition is performed while the user-facing value is still live in the old state, the harm is real:

- In Scenario A, a user can lose already-earned DEPTH that had been preserved for a later claim.
- In Scenario B, users can be left with filled legacy rounds that cannot finalize or old-contract upgrade rights that can no longer be issued.

Proof of Concept

Scenario A: challenge resolution wipes frozen rewards

1. A tier-0 vault accrues rewards.
2. Before claiming, the user calls `sacrificeDepth()` or `sacrificeToken()`, which checkpoints the current pending rewards into `unclaimedBuffer`.
3. The owner marks the vault challenged.
4. During the challenged period, `claimRewards()` returns `0` because the vault is tier 0 and `vaultStatus != 0`.
5. The user later satisfies the ticket threshold and calls `resolveChallenge()`.
6. `resolveChallenge()` clears status and resets `lastClaimTimestamp` and `unclaimedBuffer` without preserving pending rewards.
7. The previously checkpointed rewards, plus any additional rewards accrued since the last checkpoint, are erased.

Scenario B: raffle rebinding strands legacy state

1. Users buy tickets into a raffle round until it is filled, but the operator has not yet called `selectWinner()`.
2. The owner calls `setRaffleContract(newRaffle)`.
3. The operator later tries to finalize the old round through the old raffle contract.
4. The old raffle calls `setUpgradeVoucher()`.
5. The call reverts because only the new raffle contract is now authorized.
6. The old round remains stranded, and the same authorization break applies to old-contract voucher issuance paths such as `claimGuaranteedUpgrade()` and `purchaseUpgrade()`.

Recommendation

When changing protocol control-plane state, preserve or settle all user entitlements tied to the pre-transition state before finalizing the transition.

- For challenge resolution, clear the challenged status and checkpoint pending rewards before mutating `lastClaimTimestamp` or zeroing `unclaimedBuffer`.
- For raffle migration, require all legacy rounds and old-contract voucher paths to be settled before rebinding, or support a transition window where both old and new raffle contracts remain authorized.
- Add regression tests that verify administrative transitions do not erase accrued rewards or strand already-funded raffle state.

Team Response

Fixed.

[L-01] The `upgradeTier()` Uses the Vault's Frozen `totalCap` Instead of the Live Tier Config, so Lowering Any Tier Cap Permanently Bricks Upgrades for Existing Vaults

Severity

Low Risk

Description

The `setTierConfig()` and `setTierConfigs()` validate proposed tier cap changes only against adjacent config values in the `tierConfigs` mapping. Neither function checks whether the new cap would fall at or below any existing vault's frozen `vault.totalCap` — the snapshot written at initialization (`_initializeVault()`, line 728) or at the last upgrade (`upgradeTier()`, line 280; `redeemUpgradeVoucher()`, line 487).

This means a sequence of owner cap reductions — each individually passing the adjacent-config monotonicity check — can compress the tier curve below caps already stored on live vaults. Once this happens, `upgradeTier()` and `redeemUpgradeVoucher()` permanently revert with `InvalidTierCap` for those vaults, because the guard `nextCap <= currentCap` (lines 267, 480) correctly prevents an upgrade from reducing a vault's economics.

Important: `upgradeTier()` and `redeemUpgradeVoucher()` are not buggy. The `nextCap <= currentCap` check is a valid invariant — allowing it to pass when `nextCap < currentCap` would silently downgrade the vault's cap. The bug is exclusively in the setter functions, which permit config states that make that invariant unsatisfiable.

Path 1 — Sequential `setTierConfig()` calls:

The `setTierConfig()` (line 384) checks the new cap against `tierConfigs[tier - 1].cap` (the live lower-tier value) and `tierConfigs[tier + 1].cap` (the live upper-tier value). Each individual call preserves monotonicity of the config curve, but a cascade of reductions — each referencing the already-lowered neighbour — compresses the entire curve:

```
Initial config:  t0=500,  t1=1000, t2=2500
setTierConfig(1, 600)  -> check: 600 > t0(500) pass  -> t1 = 600
setTierConfig(2, 700)  -> check: 700 > t1(600) pass  -> t2 = 700

Vault at tier 1 with vault.totalCap = 1000 calls upgradeTier():
  nextCap = tierConfigs[2].cap = 700
  currentCap = vault.totalCap = 1000
  700 <= 1000 -> InvalidTierCap revert -- permanently bricked
```

Path 2 — Single `setTierConfigs()` call:

The `setTierConfigs()` (line 398) writes all new caps to storage first (lines 400-407), then validates the resulting curve for internal monotonicity only (lines 409-413). The post-write check has no knowledge of the live vault state:

```
// writes first -- no per-cap validation against vault state
for (uint256 i = 0; i < tiers.length; i++) {
    tierConfigs[tiers[i]].cap = caps[i];
}
// then validates config curve only
for (uint8 t = 1; t < NUM_TIERS; t++) {
    if (tierConfigs[t].cap != 0 && tierConfigs[t].cap <= tierConfigs[t -
        1].cap) {
         revert InvalidTierCap(t, tierConfigs[t].cap);
```

```
    }
}
```

A single call `setTierConfigs([1, 2], [600, 700])` produces the same bricked state as the sequential example, in one transaction.

Both paths leave affected vaults permanently stuck with no user-controlled recovery. The only remediation is owner intervention to re-raise the impacted cap above every affected vault's frozen `vault.totalCap`, which requires off-chain enumeration.

Location of Affected Code

File: EmissionController.sol#L384-L396

```
function setTierConfig(uint8 tier, uint256 cap) external onlyOwner {
    if (tier >= NUM_TIERS) revert InvalidTier(tier);
    if (cap == 0) revert InvalidTierCap(tier, cap);

    if (tier > 0 && cap <= tierConfigs[tier - 1].cap) revert
       InvalidTierCap(tier, cap);
    if (tier < NUM_TIERS - 1) {
        uint256 nextCap = tierConfigs[tier + 1].cap;
        if (nextCap != 0 && cap >= nextCap) revert InvalidTierCap(tier,
           cap);
    }

    tierConfigs[tier].cap = cap;
    emit TierConfigSet(tier, cap);
}
```

File: EmissionController.sol#L398-L414

```
function setTierConfigs(uint8[] calldata tiers, uint256[] calldata caps)
   external onlyOwner {
    if (tiers.length != caps.length) revert InvalidTierCap(0, 0);
    for (uint256 i = 0; i < tiers.length; i++) {
        uint8 tier = tiers[i];
        uint256 cap = caps[i];
        if (tier >= NUM_TIERS) revert InvalidTier(tier);
        if (cap == 0) revert InvalidTierCap(tier, cap);
        tierConfigs[tier].cap = cap;
        emit TierConfigSet(tier, cap);
    }

    for (uint8 t = 1; t < NUM_TIERS; t++) {
        if (tierConfigs[t].cap != 0 && tierConfigs[t].cap <= tierConfigs[
           t - 1].cap) {
            revert InvalidTierCap(t, tierConfigs[t].cap);
        }
    }
}
```

Downstream functions that become permanently bricked (these are NOT buggy — they correctly enforce the upgrade-must-not-reduce-cap invariant):

File: EmissionController.sol#L264-L267

```
function upgradeTier(uint256 tokenId) public whenNotPaused {
  // code
  uint8 nextTier = currentTier + 1;
  uint256 nextCap = tierConfigs[nextTier].cap;
  uint256 currentCap = vault.totalCap;
  if (nextCap <= currentCap) revert InvalidTierCap(nextTier, nextCap);
  // code
}
```

File: EmissionController.sol#L478-L480

```
function redeemUpgradeVoucher(uint256 tokenId) external whenNotPaused {
  // code
  uint256 nextCap = tierConfigs[nextTier].cap;
  uint256 currentCap = vault.totalCap;
  if (nextCap <= currentCap) revert InvalidTierCap(nextTier, nextCap);
  // code
}
```

Impact

- Any tier cap reduction that causes `tierConfigs[nextTier].cap <= vault.totalCap` for existing vaults makes both `upgradeTier()` and `redeemUpgradeVoucher()` permanently uncallable for those vaults — a liveness failure with no user-controlled recovery.
- Already-issued upgrade vouchers targeting an affected tier become permanently unredeemable.
- Because `vault.totalCap` differs per vault depending on when it last upgraded, a single cap change can leave some users upgradable and others not, with no uniform outcome and no on-chain visibility into which vaults are affected.
- Impact is Medium (permanent loss of progression with no self-service recovery), but likelihood is Low (requires owner cap reconfiguration). This lands at **Low** on the severity matrix.

Proof of Concept

1. Protocol deploys with tier config: `t0=500, t1=1000, t2=2500`.
2. User A initializes a vault at tier 0 (`vault.totalCap = 500`), upgrades to tier 1 (`vault.totalCap = 1000`).
3. User B initializes a vault at tier 0, upgrades to tier 1 (`vault.totalCap = 1000`). User B also receives an upgrade voucher for tier 2.
4. Owner calls `setTierConfig(1, 600)` — passes: `600 > tierConfigs[0].cap (500)`.
5. Owner calls `setTierConfig(2, 700)` — passes: `700 > tierConfigs[1].cap (600)`.
6. User A calls `upgradeTier()`:
   - `nextCap = tierConfigs[2].cap = 700`

- `currentCap = vault.totalCap = 1000`
- `700 <= 1000` -> **reverts with** `InvalidTierCap` — permanently bricked.

7. User B calls `redeemUpgradeVoucher()`:
   - Same check: `700 <= 1000` -> **reverts** — voucher permanently unredeemable.
8. No user-controlled recovery exists. Only the owner can fix this by re-raising tier 2's cap above 1000.

Recommendation

1. **Keep the** `nextCap > vault.totalCap` **invariant** in `upgradeTier()` and `redeemUpgradeVoucher()` — these are correct.
2. **Fix the setter functions** to validate against live vault state:
   - Before accepting a new cap for tier `t`, require it to remain above the maximum stored `vault.totalCap` of all vaults currently at tier `t - 1` (otherwise those vaults can never advance into tier `t`).
   - For same-tier consistency, also require the new cap to be no lower than the maximum `vault.totalCap` already assigned to vaults at tier `t`, or explicitly document grandfathering behavior.
3. **For batch updates** (`setTierConfigs()`), validate the full proposed curve and the live-vault constraints before mutating storage — currently it writes first and validates config-only afterwards.
4. **If cap reductions are operationally intended**, implement an explicit migration/reconciliation path (e.g., a function that adjusts affected vaults' `totalCap` to the new config value with appropriate safeguards) instead of silently stranding users.

Team Response

Fixed.

our shielding · Your smart contracts, our shielding · Your smart c

shieldify

Thank you!